VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Aaryan Dwivedi	24BCE11250
Slot	**Date of Submission**
B22	30-03-2026

1. Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones — was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project asks you to stop and think about that.

You will choose one real open-source software project — something you can download, read about, and interact with on Linux — and conduct a structured audit of it. Not just 'what does it do,' but why it exists, who built it, what freedoms its license actually gives you, what philosophical values it represents, and how it has shaped the world around it.

Alongside the written report, you will write five shell scripts that demonstrate practical Linux skills. These are not meant to be impressive programs — they are meant to show that you understand how open-source tools work at the command line, and how automation connects to the philosophy of sharing and transparency.

2. Choosing Your Software

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Git _____

3. Project Report Structure

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

Before Git was developed, managing source code in large software projects was a complex and inefficient process. Developers relied on centralized version control systems such as CVS and Subversion, where all code changes were stored on a single central server. This approach created several limitations, including slow performance, lack of flexibility, and dependency on a continuous network connection. If the central server failed, it could disrupt the entire development workflow.

A major turning point occurred during the development of the Linux kernel, one of the largest open-source projects in the world. Initially, the kernel developers used a proprietary version control system called BitKeeper. While BitKeeper offered advanced features, it was not fully open source, and its licensing restrictions created tension within the open-source community.

In 2005, after a dispute between the Linux developers and the BitKeeper company, access to the tool was withdrawn. This left the Linux community without a reliable version control system. As a result, Linus Torvalds decided to build a new system from scratch that would meet the needs of large-scale distributed development while remaining completely open source.

Linus Torvalds designed Git with specific goals in mind: high performance, strong data integrity, and full support for distributed workflows. Unlike traditional systems, Git allows every developer to have a complete copy of the repository, including its full history. This eliminates dependence on a central server and enables faster operations, offline work, and better collaboration among developers.

Another important feature of Git is its ability to handle branching and merging efficiently. Developers can work on multiple features simultaneously without affecting the main codebase, which significantly improves productivity in large teams.

The decision to make Git open source ensured that it could be freely used, modified, and improved by the global developer community. Over time, Git has become the backbone of modern software development, powering platforms like GitHub and enabling collaborative development at a massive scale.

In conclusion, Git was created to solve the limitations of existing version control systems and to provide a fast, reliable, and distributed solution for managing code. Its origin reflects the practical needs of developers and the principles of open-source collaboration, making it one of the most important tools in the software industry today.

A2. The license — what it actually says

Git is released under the GNU General Public License version 2 (GPL v2), which is one of the most widely used open-source licenses. This license is designed to guarantee that software remains free in terms of freedom, not just cost, and ensures that users have the right to study, modify, and distribute the software.

The concept of free software is based on four essential freedoms. The first freedom is the ability to run the program for any purpose without restrictions. The second freedom allows users to study how the program works and modify it according to their needs, which requires access to the source code. The third freedom is the ability to redistribute copies so others can benefit from the software. The fourth freedom allows users to distribute modified versions, enabling the community to improve and evolve the software over time.

Git’s GPL license grants all four of these freedoms. However, it also introduces an important

condition: if a user modifies Git and distributes the modified version, they must also release their changes under the same GPL license. This ensures that the software and any improvements to it remain open and accessible to everyone. This concept is often referred to as “copyleft,” which prevents proprietary ownership of modified versions.

A company can use Git internally or even sell a product that includes Git, but if they distribute a modified version of Git itself, they are required to share the source code of those modifications. This restriction protects the open-source nature of the software while still allowing commercial use.

The GPL license differs significantly from more permissive licenses such as the MIT License. The MIT License allows developers to modify and distribute software, even as part of proprietary products, without the obligation to share their changes. In contrast, GPL ensures that all derivative works remain open source. If I were to choose a license for my own project, I would prefer GPL if I wanted to ensure that my work and its improvements remain open, and MIT if I wanted maximum flexibility for commercial use.

There is also an important distinction between “free as in free beer” and “free as in freedom.” “Free as in free beer” means that the software is available at no cost, while “free as in freedom” refers to the user’s rights to control, modify, and share the software. Git represents “free as in freedom” because it not only costs nothing but also gives users complete control over how they use and modify it.

A3. The ethics of open source

Open source is not only a development model but also a philosophy about how knowledge and technology should be shared. It promotes transparency, collaboration, and collective improvement. However, the question of whether all software should be open source is complex and requires a balanced perspective.

open-source software offers significant advantages. It allows anyone to study, modify, and improve the code, which leads to faster innovation and better security. Since the source code is publicly available, vulnerabilities can be identified and fixed by a global community of developers. In the case of Git, its open nature has enabled millions of developers worldwide to collaborate efficiently, making it a cornerstone of modern software development.

not all software can or should be open source. Some companies invest large amounts of time and resources into developing software and rely on proprietary models to recover costs and generate profit. In certain cases, keeping software closed can also enhance security by limiting access to sensitive systems. Therefore, a balance between open-source and proprietary software is necessary depending on the context and purpose.

Another important ethical question is whether it is fair for companies to build profitable products on top of open-source software. Many large organizations use open-source tools like Git and contribute back to the community, but others may benefit without giving back. While it is legally allowed, it raises ethical concerns about fairness and sustainability. Ideally, companies should contribute improvements, funding, or support to the projects they rely on, ensuring that the ecosystem remains healthy.

The phrase “standing on the shoulders of giants” is highly relevant in open source. It means that developers build new innovations on top of existing work created by others. Open source strongly enables this process by making knowledge freely available. Rather than limiting innovation, it accelerates it by allowing developers to focus on new ideas instead of reinventing existing solutions.

However, there is also an argument that excessive reliance on existing open-source tools may reduce original thinking. Developers might depend too heavily on pre-built solutions instead of understanding underlying concepts. Therefore, while open source promotes innovation, it also requires responsible usage and continuous learning

Part B — Linux Footprint (Unit 2)

1. Installation Method
Git is widely available in Linux distributions and can be installed using standard package managers such as apt (Debian/Ubuntu-based systems) or rpm/dnf (Red Hat-based systems). On Ubuntu, Git can be installed using the following command:

```
sudo apt install git
```

2. Key Directories
After installation, Git is distributed across several standard Linux directories:

- **Binary file:** /usr/bin/git
- **Configuration files:**
  - System level: /etc/gitconfig
  - User level: ~/.gitconfig
- **Repository data:**
  Each Git repository contains a hidden .git directory that stores:
  - Commit history
  - Branches
  - Configuration

3. User and Group
Git does not run as a dedicated system service or daemon. Instead, it operates under the current logged-in user.

Why this matters for security:

- Access control depends on file permissions
- Only authorized users can modify repositories
- Prevents unauthorized system-wide changes

4. Service Management
Git is not a background service, so it cannot be started or stopped using systemctl like server software.
Instead, Git is executed as a command-line tool:
git --version
git init
git clone <repository-url>
However, Git can be used with services like:

- Git servers (e.g., SSH-based access)
- Platforms like GitHub or GitLab

5. Update Model
Git is maintained by an open-source community led by contributors worldwide. Updates and security patches are released regularly and distributed through Linux package repositories.
Users can update Git using:
sudo apt update
sudo apt upgrade git
This ensures that:

- Security vulnerabilities are patched quickly
- Users receive stable and tested versions
- Updates are integrated into the system through trusted repositories

Part C — The FOSS Ecosystem (Units 3 & 4)

1. Dependencies and Supporting Tools
Git, although a standalone version control system, relies on several open-source tools and system components to function efficiently. It is primarily written in C, so it depends on standard system libraries such as the GNU C Library (glibc). It also uses tools like OpenSSL for secure communication over HTTPS and SSH for secure remote repository access.
Additionally, Git integrates with text editors such as Vim and Nano for commit messages, and it can work with scripting languages like Python and Perl for hooks and extensions. These dependencies highlight how Git operates within a broader open-source environment rather than in isolation.

2. Tools Enabled and Inspired by Git
Git has enabled a massive ecosystem of tools and platforms built on top of it. One of the most important is GitHub, which provides cloud-based repository hosting and collaboration features. Similarly, platforms like GitLab and Bitbucket offer integrated development environments, CI/CD pipelines, and team collaboration tools.
Git has also inspired graphical user interfaces (GUIs) such as GitKraken and Sourcetree, making version control more accessible to beginners. Furthermore, it has influenced modern DevOps practices, enabling continuous integration and continuous deployment workflows.
The concept of distributed version control introduced by Git has reshaped how developers collaborate globally, allowing multiple contributors to work independently and merge their changes efficiently.

3. Relation to the Web and LAMP Stack
Git is not directly part of the traditional LAMP stack (Linux, Apache, MySQL, PHP), but it plays a critical supporting role in web development. Developers use Git to manage source code for web applications that run on the LAMP stack.
For example:

- Code written in PHP is version-controlled using Git
- Updates to web applications are tracked and deployed using Git workflows
- Teams collaborate on web projects through Git repositories

In this way, Git acts as the backbone of development workflows, even though it is not a runtime component of web servers.

4. Community and Governance
Git is supported by a large and active open-source community. The project was originally created by Linus Torvalds, but it is now maintained by contributors worldwide.
The development of Git is coordinated through mailing lists, particularly the Git mailing list, where developers discuss updates, report bugs, and review code. Contributions are peer-reviewed before being accepted, ensuring high-quality development standards.
There is no single corporate owner controlling Git. Instead, it follows a merit-based governance model where experienced contributors and maintainers guide the project. This decentralized governance aligns with open-source principles and ensures transparency.
In addition, Git has a strong global community presence through forums, online documentation, conferences, and developer meetups. This widespread support system helps both beginners and advanced users effectively use and contribute to the tool.

Part D — Open Source vs Proprietary (Critical Analysis)

Dimension	Open Source	Proprietary Alternative
Cost	Free to use, no license cost	Paid license required
Security (who can audit the code?)	Anyone can audit the code	Only company can audit
Support and reliability	Community support, large documentation	Official company support available
Freedom to modify	Full freedom to modify and share (GPL)	No modification allowed
Community vs corporate control	Community-driven development	Controlled by single company
Your overall verdict	More flexible, open, and widely used	Limited flexibility, better for enterprise support

Git is a highly reliable and efficient version control system that has become essential in modern software development. Its open-source nature provides transparency, flexibility, and strong community support, making it suitable for both individual developers and large organizations. The ability to track changes, manage branches, and collaborate seamlessly gives Git a significant advantage in real-world deployment. Based on its performance, scalability, and widespread adoption, I would strongly recommend Git for practical use in software projects.

At the same time, proprietary tools like BitKeeper offer structured support and controlled environments, which may be beneficial for certain enterprise requirements. However, they lack the openness and freedom that Git provides. In my opinion, Git strikes the right balance between functionality and flexibility, and I would be interested in contributing to its ecosystem in the future, whether through usage, learning, or community participation.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Starter structure

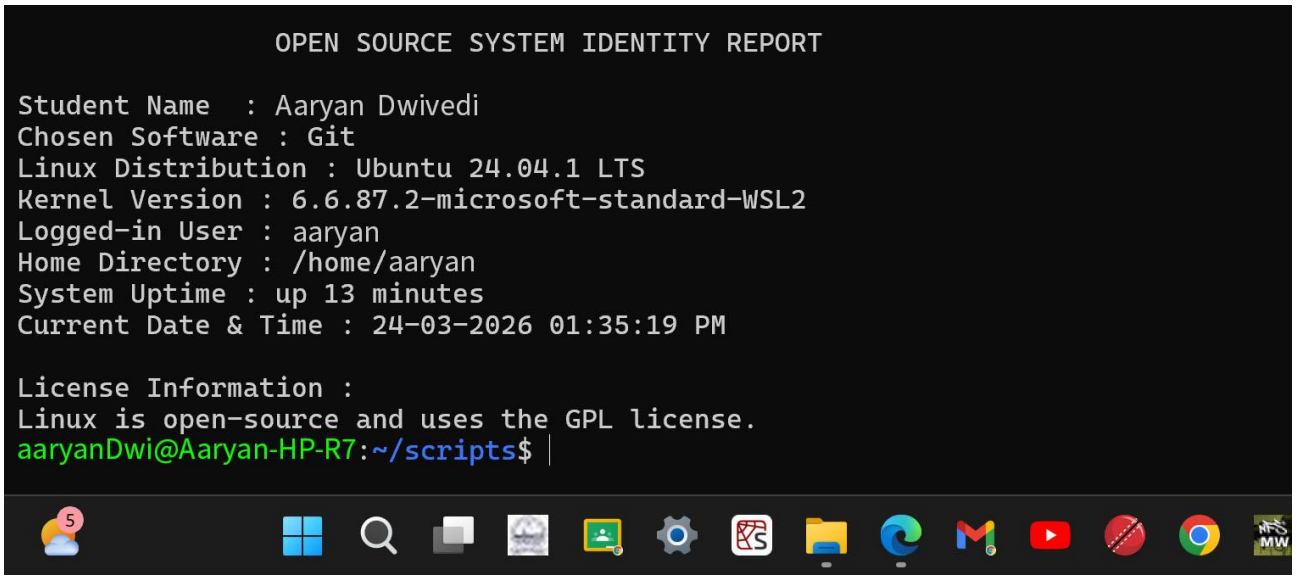
```
#!/bin/bash
# Script 1: System Identity Report
# Author: Aaryan Dwivedi | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Aaryan Dwivedi "
SOFTWARE_CHOICE="Git"

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)

# --- Display ---
echo "================================"
echo "  Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Uptime  : $UPTIME"
# TODO: Add distro name, date, and license message
```

Output:



Script 2 — FOSS Package Inspector (Unit 2)

Concepts to use: if-then-else, case statement, rpm -qi or dpkg -l, pipe with grep.

Starter structure

```
#!/bin/bash
# Script 2: FOSS Package Inspector


PACKAGE="Git"



#we will check if package is installed or not
if dpkg -l | grep -w "$PACKAGE" > /dev/null 2>&1; then
   echo "$PACKAGE is installed."
   dpkg -l | grep -w "$PACKAGE"
   echo "Version information:"
   git --version
else
   echo "$PACKAGE is NOT installed."
fi

echo ""

#using case statement for package description
case $PACKAGE in
   git)
      echo "Git: an open-source version control system created to track code changes
efficiently.";;
   apache2)
      echo "Apache: the web server that helped build the open internet.";;
   mysql-server)
      echo "MySQL: an open-source database used in many real-world applications.";;
   firefox)
      echo "Firefox: an open-source browser that supports an open and accessible
web.";;
   vlc)
      echo "VLC: an open-source media player that can play almost any audio or video
format.";;
   *)
      echo "No philosophy note available for this package.";;
esac
```

Output:

```
aaryanDwi@Aaryan-HP-R7:~/scripts$ nano script2.sh
aaryanDwi@Aaryan-HP-R7:~/scripts$ chmod +x script2.sh
./script2.sh
git is installed.
ii  git                         1:2.43.0-1ubuntu7.1                   amd64        fast, scalable, distributed revision control system
ii  git-man                     1:2.43.0-1ubuntu7.1                   all          fast, scalable, distributed revision control system (manual p
ages)
ii  lshw                        02.19.git.2021.06.19.996aaad9c7-2build3 amd64        information about hardware configuration
Version information:
git version 2.43.0

Git: an open-source version control system created to track code changes efficiently.
```

Script 3 — Disk and Permission Auditor (Unit 2)

Concepts to use: for loop, df, ls -ld, awk or cut to extract fields.

Starter structure

```
#!/bin/bash

#Script-3: Disk and Permission Auditor
#Author:Aaryan Dwivedi
#Course:Open Source Software

#list of directories to check
DIRS=("/etc" "/var/log" "/home" "/usr/bin"
"/tmp")

echo " "
echo "      Directory Audit Report        "
echo " "

#doing loop through directories
for DIR in "${DIRS[@]}"; do
   if [ -d "$DIR" ]; then
      PERMS=$(ls -ld "$DIR" | awk '{print $1, $3,
$4}')
      SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
      echo "$DIR => Permissions: $PERMS | Size:
$SIZE"
   else
      echo "$DIR does not exist"
   fi
done

echo " "
echo "Checking Git configuration directory"
echo " "

#checking git config directory
if [ -d "$HOME/.git" ]; then
   ls -ld "$HOME/.git"
else
   echo "Git configuration directory not found in
home"
fi
```

Output:

```
aaryanDwi@Aaryan-HP-R7:~/scripts$ nano script3.sh
aaryanDwi@Aaryan-HP-R7:~/scripts$ chmod +x script3.sh
./script3.sh

        Directory Audit Report

/etc => Permissions: drwxr-xr-x root root | Size: 4.2M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 73M
/home => Permissions: drwxr-xr-x root root | Size: 60K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 92M
/tmp => Permissions: drwxrwxrwt root root | Size: 20K

Checking Git configuration directory

Git configuration directory not found in home
```

Script 4 — Log File Analyzer (Units 2 & 5)

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

Starter structure

```
#!/bin/bash

#Script-4: Log File Analyzer
#Author:Aaryan Dwivedi
#Course:Open Source Software

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

#checking if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

#checking if file is empty
if [ ! -s "$LOGFILE" ]; then
    echo "The file is empty."
    exit 1
fi

#reading file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output:

```
aaryanDwi@Aaryan-HP-R7:~$ nano script4.sh
aaryanDwi@Aaryan-HP-R7:~$ chmod +x script4.sh
./script4.sh
Error: File  not found.
aaryanDwi@Aaryan-HP-R7:~$ ./script4.sh script2.sh git
Error: File script2.sh not found.
aaryanDwi@Aaryan-HP-R7:~$ cd scripts
./script4.sh script2.sh git
Keyword 'git' found 4 times in script2.sh

Last 5 matching lines:
PACKAGE="git"
    git --version
    git)
        echo "Git: an open-source version control system created to track code changes efficiently.";;
aaryanDwi@Aaryan-HP-R7:~/scripts$
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

Starter structure

```
#!/bin/bash

#Script 5: Open Source Manifesto
Generator
#Author: Aaryan Dwivedi
#Course: Open Source Software

echo "Answer three questions to
generate your manifesto."
echo ""

read -p "1.name one open-source
tool you use every day: " TOOL
read -p "2.in one word, what does
freedom mean to you? " FREEDOM
read -p "3.name one thing you would
build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

#example alias: alias ll='ls -l'

#writing manifesto to file
echo "Open Source Manifesto" >
"$OUTPUT"
echo "Date: $DATE" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "I use $TOOL in my daily life,
and it reminds me that open source
is not only about code but also
about $FREEDOM.If i have the
opportunity,i would build and
freely share $BUILD so that others
could learn, improve it, and
benefit from it. I believe open
source encourages collaboration,
creativity,and equal access to
technology for everyone." >>
"$OUTPUT"
echo " "
echo "Manifesto saved to $OUTPUT"
echo " "
cat "$OUTPUT"
```

Output:

```
aaryanDwi@Aaryan-Hp-R7:~/scripts$ nano script5.sh
aaryanDwi@Aaryan-Hp-R7:~/scripts$ chmod +x script5.sh
./script5.sh
Answer three questions to generate your manifesto.

1.name one open-source tool you use every day: git
2.in one word, what does freedom mean to you? choice
3.name one thing you would build and share freely: software app

Manifesto saved to manifesto_aaryan.txt

Open Source Manifesto
Date: 24 March 2026

I use git in my daily life, and it reminds me that open source is not only about code but also about choice.If i have the opportunity,i would build
and freely share software app so that others could learn, improve it, and benefit from it. I believe open source encourages collaboration, creativit
y,and equal access to technology for everyone.
aaryanDwi@Aaryan-Hp-R7:~/scripts$
```